ARCHITECTURE SCALABILITY BLUEPRINT

MonthlyGrocery Architecture & Bottleneck Audit

Structural scalability roadmap for handling 10,000+ simultaneous customers, 300+ Kirana merchants, and 44,000 product SKUs without system freezes, race conditions, or connection timeouts.

Target Volume:	10,000 DAU Customers
Active Stores:	300+ Kirana Shops
Catalog Target:	44,000 Master SKUs
Infra Target:	AWS RDS PostgreSQL + S3

Local File I/O Lock

`db.json`

Synchronous `fs.writeFileSync` blocks Node.js event loop during concurrent checkout surges.

DB Connection Pool

`20 Max`

PostgreSQL connection pool max 20 causes queue starvation under 10k users peak checkout load.

Merchant Real-time

`HTTP Polling`

Inventory Concurrency

`Non-Atomic`

300 merchants polling every 5s generates ~3,600 req/min of unnecessary database load.

Check-then-deduct pattern in JS memory allows overselling the last stock item to multiple users.

System Concurrency & Data Flow Map

Current dual-write pattern vs. recommended high-throughput enterprise architecture



1. Synchronous File Locking & Dual-Write Drift (`db.json`)

`CRITICAL • HIGH RISK`

The Issue: Across `routes/orders.ts`, `routes/shops.ts`, and `routes/addresses.ts`, every checkout or status change reads and writes to a single disk file (`data/db.json`) using `fs.writeFileSync()` alongside writing to PostgreSQL.

Why It Fails at 10k Scale: Node.js is single-threaded. Serializing a 10MB JSON file to disk blocks the entire event loop for 100ms+. When 50 customers check out at the same time, simultaneous file writes overwrite each other (causing lost orders) and freeze the server for other customers.

Recommended Solution: Completely decommission `db.json` for transaction and order data. Use AWS RDS PostgreSQL with standard ACID transactions (`BEGIN ... COMMIT`) as the sole source of truth.

2. Database Connection Pool Starvation (`max: 20`)

`HIGH • CONNECTION TIMEOUT`

The Issue: In `express-backend/src/config/db.ts`, the PostgreSQL connection pool is configured with `max: 20` and `connectionTimeoutMillis: 10000`.

Why It Fails at 10k Scale: When 1,000 concurrent customers browse catalog pages, apply coupons, and checkout while 300 merchants poll their order screens, all 20 connections become occupied. Incoming requests queue up and eventually fail with `Connection timeout` errors.

Recommended Solution: Enable **AWS RDS Proxy** to pool and multiplex database connections seamlessly up to hundreds of concurrent workers, and increase Node pool size to **60–80**.

3. In-Memory Variant Grouping Overhead for 44,000 SKUs

`HIGH • MEMORY & CPU CHURN`

The Issue: `groupProductsByFamily` fetches hundreds of individual product rows into Node memory and groups them by family using JavaScript `Map` and regex replacements.

Why It Fails at 10k Scale: As the catalog reaches 44,000 SKUs, running CPU-intensive string replacements and array grouping on every API request consumes server RAM and causes Garbage Collection (GC) pauses.

Recommended Solution: Create a PostgreSQL SQL View or Materialized View with `json_agg(variants)` so PostgreSQL aggregates variants at the database level in under 10ms, returning clean pre-grouped JSON.

```
CREATE OR REPLACE VIEW view_grouped_products AS SELECT family_key, brand, MIN(price) as starting_price, json_agg(json_build_object('id', id, 'sku', sku, 'unit', unit, 'price', price, 'stock', stock)) as variants FROM products GROUP BY family_key, brand;
```

4. Inventory Race Conditions (Overselling Last Packets)

`HIGH • BUSINESS INTEGRITY`

The Issue: Currently, stock availability is checked in JavaScript memory during checkout, and stock is deducted later in a separate asynchronous call (`deductShopInventory`). There is no row-level database lock.

Why It Fails at 10k Scale: If a merchant has only 2 units of Amul Butter left and 3 customers checkout in the same second, all 3 pass the stock validation, leading to overselling and merchant cancellation.

Recommended Solution: Use atomic SQL update queries with conditional stock guards:

```
UPDATE shop_products SET stock = stock - $1, updated_at = NOW() WHERE shop_id = $2 AND product_id = $3 AND stock ⩾ $1 RETURNING stock;
```

5. Merchant Order Notification: HTTP Polling vs. Push Events

`MEDIUM • EFFICIENCY`

The Issue: `OrdersDashboard.tsx` relies on screen refocus and polling to detect new orders. 300 merchants actively polling causes thousands of database queries every minute.

Recommended Solution: Implement **Firebase Cloud Messaging (FCM)** push notifications with high-priority order chimes, paired with a lightweight **WebSocket / Socket.io** connection for instant live dashboard updates without continuous polling.

Scalability Implementation Roadmap

Phase-wise Plan

Phase 1: Immediate Lockdown (Sprint 1)

- Migrate all order writes exclusively to AWS RDS PostgreSQL.
- Implement atomic SQL inventory decrement.
- Increase DB pool to 60 connections.

Phase 2: High Throughput (Sprint 2)

- Integrate Redis caching for catalog & store statuses.
- Deploy FCM push notifications for merchant orders.
- Create SQL view for variant aggregation.

Phase 3: Production Scale (Sprint 3)

- Deploy AWS RDS Proxy for connection pooling.
- Run Express backend in Docker on AWS ECS Fargate with ALB.
- Setup CloudWatch metrics for latency and connection alerts.